

Causal Models

BIOL 3401 Mount Royal University

Topic 8, Winter 2026

Contents

Making sense of your models	1
Learning Objectives	1
Set up your R workspace for the day	2
Part 1: Construct DAGs	2
Part 2: Correlations	3
Part 3: Multicollinearity	4
Part 4: Post-treatment bias	5
Part 5: Collider bias	5
Part 6: Making unbiased models for more complex DAGs	6
Part 7: Overfitting	7
Part 5: Standardizing	9

Making sense of your models

Today, we are going to learn some important multiple regression decision-making tools. Whether we are interested in causal or predictive modelling, we need to think carefully about the variables we include in our models (or don't).

Learning Objectives

After completing this topic, you should feel comfortable:

1. Constructing Directed Acyclical Graphs (DAGs) representing your model worlds
2. Identifying biases from DAGs
3. Choosing the correct variables to include in your unbiased multiple regression models
4. Explaining how overfitting arises and how it impacts predictions made on new data
5. Standardizing your multiple-regression models for interpretation

Set up your R workspace for the day

Set your working directory

Make sure we're setting our working directory!

```
setwd()
```

Install and load packages

We are going to need to load **tidyverse** and **DHARMA**. We are also going to install and load a new package called **dagitty**, which will help us visualize DAGs for our model systems.

Using the code block below, install the new package and load the packages.

Load data

- Today we will use three datasets to explore how multiple regression can produce misleading results if we are not careful about which variables we include in our models.
- **crop_yields.csv**: Contains records of annual wheat harvest yields (in bushels/acre) across 60 farms from 2021-2025 according to weather conditions the previous winter. Variables include:
 - **snow_cm**: The total amount of snowfall (cm)
 - **temp_C**: The average winter temperature (in °C)
 - **yield_ba**: The wheat yield after harvest, in bushels/acre
- **tomatoes.csv**: Includes data on fruit production by tomato plants. Variables include:
 - **fertilizer_mg**: The amount of fertilizer applied to the plant's soil (mg)
 - **plant_height_cm**: The plant's height (cm)
 - **fruit_count**: The number of fruits (tomatoes) produced by the plant
- **wine_quality.csv**: Data on the quality and price of wine bottles. Variables include:
 - **quality_score**: Assessment of the wine bottle made by a critic on a 10-point scale
 - **price_CAD**: The price of the bottle in Canadian dollars
 - **acclaim**: The critical acclaim of the winery that produced the bottle, ranked on a 100-point scale

Load the datasets by finishing the `read_csv()` functions.

```
crop_yields <- read_csv()
wine_quality <- read_csv()
tomatoes <- read_csv()
```

Part 1: Construct DAGs

We will shortly test for relationships between the variables in our models to decide which biases we need to control for. However, it's best to start our modelling by building an intuitive conceptual model of how our model system works. We will use **dagitty** to construct our conceptual models, starting with the **crop_yields** data.

Constructing our first DAGs

In our fictional world of crop yields:

- Lower temperatures the previous winter increase wheat yields.
- Heavier snowfall the previous winter increases wheat yields.
- Winters with warmer temperatures tend to have less snowfall.

We construct a DAG using the `dagitty()` function. The arguments are the relationships between the variables. Each suspected relationship between two variables is described with the variable names, separated by a `->`. These are each entered on a new line *without a separating comma*. Two-way relationships need to be specified on individual lines. We can then plot the object we create using `plot()`.

```
# Make the DAG object
crop_DAG <- dagitty(" {
Temp -> Yield
Snow -> Yield
Temp -> Snow
Snow -> Temp
} ")

# Draw the DAG
plot(crop_DAG)
```

We can similarly describe conditions in our fictional tomato and wine quality worlds:

In tomato world:

- Fertilizer increases plant growth
- Bigger plants produce more fruit

In wine world:

- Bottle price increases the critical acclaim score of the winery
- Quality increases the critical acclaim score of the winery

Mini Challenge 1

1. Use `dagitty()` to construct DAGs for the tomato and wine worlds.
2. Call the DAGs `tomato_DAG` and `wine_DAG`
3. Plot the DAGs to visualise them.

Use the code block below for your answer!

```
#-----
```

Part 2: Correlations

Now that we've visualized the relationships between our variables, we're going to check whether and to what extent they are actually associated. The function `cor()` creates a correlation matrix that displays all the pair-wise correlations between variables in the data frame or tibble we supply as its only argument.

The correlation matrix can tell us whether our intuitions about our model world are correct. First, let's plot our crop DAG and check the associations.

```
plot(crop_DAG)
cor(crop_yields)
```

The resulting matrix shows the associations between all variables. Correlations range from zero to one, with one being a perfect correlation. The diagonal of the matrix shows the relationship between each variable and itself—this should always equal one. The off-diagonals show the associations between different variables. Because there is no implication of direction (this is just an association), the correlation matrix is symmetrical on either side of the diagonal.

Mini Challenge 2

Check the variable associations implied in your tomato and wine DAGs. Do they also imply the DAGs are correct?

Use the code block below for your answer!

```
#-----
```

Part 3: Multicollinearity

We are going to explore how multicollinearity, post-treatment bias, and collider bias can complicate our model interpretations when we aren't careful about framing our causal models. We will start with multicollinearity.

Test the unbiased relationship between snow depth and wheat yields

For our first test, we are going to use our `crop_yields` data to test whether *snow depth decreases wheat yields*.

We just saw that snow depth is highly negatively correlated with temperature; in warmer years (higher temperature), there is less snow. Thus, we have a multicollinearity problem. From lecture, we know that our best recourse in this situation is to *test the effect of snow depth on wheat yields independently of temperature*.

Yield and snow depth are both continuous variables that we might expect to meet the assumptions of a simple linear regression. Let's fit the model and quickly test our assumptions.

```
yield_snow_m <- lm(yield_ba ~ snow_cm, data = crop_yields)
plot(yield_snow_m)
```

Maybe a bit of skewness in the residuals, but not bad. Let's proceed.

```
summary(yield_snow_m)
```

How would we interpret the effect of snow depth on bushels per acre of wheat?

Test the relationship with multicollinearity

We saw that snow depth and temperature are highly correlated. We think we've constructed the correct model by including only snow depth and not temperature in the model. Let's see what happens when we include both variables in the same model, *testing the effects of both snow depth and temperature on wheat yields in multiple regression*.

```
yield_snow_temp_m <- lm(yield_ba ~ snow_cm + temp_C, data = crop_yields)
summary(yield_snow_temp_m)
```

The effects of snow depth and temperature have disappeared! Notice, however, that the adjusted R-squared is *slightly higher* than the model including just snow depth. This would not be a terrible model if our goal was to predict wheat yields, although it likely wouldn't be a "better" model in terms of AIC (you can check this yourself!). The big issue is that snow depth and temperature explain roughly the same amount of variation in the response, so they essentially cancel each other out when we include both variables in the same model.

Part 4: Post-treatment bias

Now let's see how post-treatment bias might impact our model interpretations, *testing whether fertilizer increases tomato production*.

Mini Challenge 3

Using your DAGs and correlation matrices for reference, test whether fertilizer increases tomato production.

1. Fit the *unbiased* model and print the model summary to evaluate the results.
2. Fit a model *including the post-treatment bias* and print the model summary to evaluate the results.

Be sure to use the appropriate model type for the data, and test your assumptions!

Use the code block below for your answer!

```
#-----
```

In post-treatment bias, the intermediate variable explained all of the variation between the response and our variable of interest. A somewhat related bias, which we won't discuss in much detail, is called *omitted variable bias*. In omitted variable bias, we mistakenly attribute the explained variation to our variable of interest, when, in fact, an "omitted" upstream variable is responsible. Our biased model for explaining the effect of fertilizer on the number of tomatoes would in fact be the *unbiased* model if we were instead interested in explaining the effect of plant height on tomato number. Hopefully, this example makes it very clear that *whether your model is appropriate depends entirely on what question you are asking!*

Part 5: Collider bias

For our last example, we are going to test *whether higher-quality wine is higher-priced*.

Mini Challenge 4

Using your DAGs and correlation matrices for reference, test whether wine quality increases the price.

1. Fit the *unbiased* model and print the model summary to evaluate the results.
2. Fit a model *including the collider bias* and print the model summary to evaluate the results.

Be sure to use the appropriate model type for the data, and test your assumptions!

Use the code block below for your answer!

```
#-----
```

Part 6: Making unbiased models for more complex DAGs

Multicollinearity, collider bias, and post-treatment bias are very common. However, your DAGs might be more complex, including multiple biases. There is a lot more to know about DAGs and causal models—this is an entire field of math and statistics! For now, it's enough to know that you need to be careful in constructing your models. Fortunately, there are shortcuts for identifying more complicated biases.

We are going to use a new function in `dagitty` called `adjustmentSets()`. This function *gives you the minimum set of variables you would need to include in your model to test the relationship you are interested in, excluding the variables you should not include.*

A more complicated model world

Let's construct a more complicated DAG than we've considered so far. Imagine we are interested in comparing aerobic performance across mammals.

We have the following variables (abbreviated for simplification):

- Altitude where the species lives (Alt)
- Hemoglobin concentration (Hem)
- Lung capacity (LC)
- Activity level (AL)
- Body size (BS)
- Blood oxygen saturation (O2)
- Aerobic performance (Aer)

We know the following information:

- Species living at higher altitudes tend to have higher hemoglobin concentrations
- High altitudes reduce oxygen saturation
- Larger species have higher lung capacity
- Activity increases lung capacity
- Hemoglobin concentration and oxygen saturation improve aerobic performance
- Body size increases aerobic performance

Our question is whether hemoglobin concentration affects aerobic performance.

The first thing we want to do is to identify and write out the associations between variables based on the information we know:

- Alt -> Hem -> Aer
- Alt -> O2 -> Aer
- LC -> O2 -> Aer
- AL -> LC
- BS -> LC
- BS -> Aer
- Hem -> O2

Mini Challenge 5

1. Using the associations we just identified between variables, create a new DAG.
2. Call the DAG `aerobic_DAG`.
3. Plot the DAG to view it.

Use the code block below for your answer!

```
#-----
```

This DAG is far more complicated than we've seen! We might have a difficult time identifying all of the biases we would need to account for to answer our question.

Next, let's use `adjustmentSets()` to figure out which variables we might need to include in our model *testing the effect of hemoglobin concentration on aerobic performance*. The function needs the following arguments:

- **x**: A DAG graph you create for your model system.
- **exposure**: The name of the predictor variable(s). The name(s) need to match the variables you included in your DAG.
- **outcome**: The name of the response variable, also matching one of the variables from your DAG.

Mini Challenge 6

Use `adjustmentSets()` to figure out which variables you would need to include in your model. The variables we *need* to include appear in a pair of curly braces '{}'.
Use the code block below for your answer!

```
#-----
```

Which variables do we need to include?

Part 7: Overfitting

We will now switch gears to focus more on *predictive modelling*. We discussed overfitting as an issue where our model learns about the noise in our data rather than the pattern we are trying to isolate because we have added too many meaningless variables. Although this *could* technically happen in a causal modelling context, especially if we have a very small dataset, we know enough now to try to avoid adding too many variables to our causal models!

We are going to run a simulation to understand how overfitting works. We will walk through the simulation together.

7a: Create simulated data

We are going to simulate a true relationship between a response variable and predictor. To the same tibble, we will then add another set of random variables having no association with the response variable.

```
# Setting a random 'seed' so we can all reproduce the same
# results
set.seed(10)

# Set the number of samples we want to use
n <- 200

# Create a random normal distributed variable 'x' with 200
# values, mean 5, sd 1
x <- rnorm(n, 5, 1)

# Simulate a new variable, y, which is positively
# associated with x
```

```

y <- 2 * x + rnorm(n, 0, 1)

# Combine x and y in a tibble 'dat'
dat <- tibble(y, x)

# Create 80 random variables (z1 to z80) and add them to
# 'dat'
for (i in 1:80) {
  dat[[paste0("z", i)]] <- rnorm(n, 5, 1)
}

```

7b: Split into “training” and “testing” data

In predictive modelling (e.g., machine learning), we often split the data into separate “training” and “testing” data. This process allows us to “train” a model to make good predictions, then test its abilities on a withheld “testing” dataset.

```

# Sample half the data by index to reserve as 'training'
# data
train_id <- sample(1:n, n/2)

# Isolate the training data and use the rest for 'testing'
train <- dat[train_id, ]
test <- dat[-train_id, ]

```

7c: Fit training models and predict

Now, we will fit the training models and use them to predict the testing data. If we can predict the testing data accurately, our model is likely serving its purpose. We will fit “simple” model using just the two variables we know are associated, and another “overfit” model including all the random variables we generated.

We can compare our predictions to the actual observations of our random variable using the residual sums of squares. This is a similar process we would use to manually calculate the R-squared, so we are essentially asking how much of the variation in our actual data our model explains.

```

# Fit a simple model y ~ x
m_simple <- lm(y ~ x, data = train)
summary(m_simple)

# Make a within-sample (training data) prediction using the
# simple model
pred_train_simple <- predict(m_simple, train)
# Calculate the mean squared prediction error
mean((train$y - pred_train_simple)^2)

# Make an out-of-sample (testing data) prediction using the
# simple model
pred_test_simple <- predict(m_simple, test)
# Calculate the mean squared prediction error
mean((test$y - pred_test_simple)^2)

# Fit an overfit model

```

```

m_overfit <- lm(y ~ ., data = train)
summary(m_overfit)

# Make a within-sample (training data) prediction using the
# overfit model
pred_train_overfit <- predict(m_overfit, train)
# Calculate the mean squared prediction error
mean((train$y - pred_train_overfit)^2)

# Make an out of-sample (training data) prediction using
# the overfit model
pred_test_overfit <- predict(m_overfit, test)
mean((test$y - pred_test_overfit)^2)

```

Notice the actual model R-squareds from the summary outputs. The R-squared is lower for our simple model, suggesting it leaves some variation unexplained. However, the multiple and adjusted R-squareds are similar, suggesting we are not fitting any extraneous noise in the data. Compare this R-squared to that of the overfit model. We can see that the multiple R-squared is very good, explaining almost 100% of the variation in the data. However, the adjusted R-squared has dropped significantly, telling us that we are fitting lots of noise when we penalize the model for all the extra variables!

What does the sum of squares tell us about how well the simple versus overfit model predicts patterns in the out-of-sample testing data?

7d: Plot the predictions

Here we are overlaying the predictions from our simple and overfitted models.

```

# Overlay the relationship predicted by the simple model on
# the data
plot(train$y ~ train$x)
lines(pred_train_simple ~ train$x)

# Overlay the relationship predicted by the overfit model
# on the data
plot(train$y ~ train$x)
lines(pred_train_overfit ~ train$x)

```

What can we say about the comparison between these two plots?

Part 5: Standardizing

The last thing we will do is to introduce a helpful trick for model interpretation. We saw back in Topic 5 that we could re-scale our variables to help interpret data on different scales (e.g., the log scale), where the variation in the data was obscured by the huge differences between extremely large and small values. We face a similar issue in multiple regression, where our effect sizes might be on different scales for a similar reason. In multiple regression, these vast differences can make it difficult to compare the magnitudes of two variables' effects in the same model. The solution is to *re-scale* our predictor variables using a *Z-score standardization*. This process turns our variables into Z-scores, placing them on the same scale even if they are orders of magnitude different.

Setting aside our collider bias for a moment, let's revisit a multiple regression model where we would like to *predict winery acclaim from bottle price and bottle quality using the wine_quality data*.

```
acclaim_qual_price_m <- lm(acclaim ~ price_CAD + quality_score,  
  data = wine_quality)  
summary(acclaim_qual_price_m)
```

Both variables have a strong effect on acclaim. However, the effect size appears to be much smaller for price than quality. Does this mean that price has less impact on acclaim than quality? Or does quality appear to have a stronger impact because it varies over a much narrower range?

To investigate, we first need to standardize both predictor variables. We will use the `scale()` function, which takes as its only argument the vector or data frame column we would like to standardize.

CAUTION: After standardizing your variables, your variable units will have changed. After that, you will need to interpret them on the Z-score scale you specified. I.e., each *standard deviation* increase in the predictor variable X now results in some ## change in the response variable Y.

Mini Challenge 7

1. Add two new variables to your `wine_quality` tibble containing the standardized versions of `price_CAD` and `quality_score`.
2. Call the new variables `price_CAD_z` and `quality_score_z`.
3. Re-model the `acclaim ~ price + quality` relationship using the new standardized variables.

Use the code block below for your answer!

```
#-----
```

Given your results, which variable do you think has a greater impact on winery acclaim?